

编译原理大作业 1 & 2

类 Rust 语言的词法/语法分析、语义分析与四元式中间代码生成

2351078 李堂辉

2351837 高胜寒

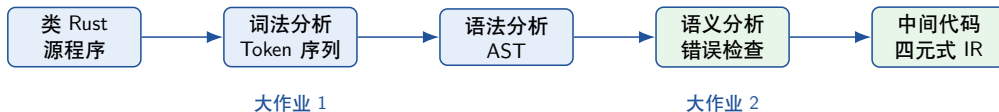
2352197 冉子易

(按学号为序)

同济大学

2026 年 6 月 16 日

汇报主线：两个作业是一条编译流程



阶段	核心问题	产物
大作业 1	源码是否符合词法/语法规则	Token、AST
大作业 2	AST 是否语义正确，如何线性化	语义诊断、四元式 IR

核心文件地图

位置	文件	职责
大作业 1	token.rs	Token 类型、显示格式、类别划分
大作业 1	lexer.rs	字符扫描、关键字识别、注释跳过、错误定位
大作业 1/2	parser.rs, ast.rs	递归下降解析、AST 节点定义
大作业 2	compiler.rs	符号表、语义检查、AST 到 IR 的翻译
大作业 2	ir.rs	Op / Arg / Quadruple / IRProgram

大作业 1 的目标与范围

输入

类 Rust 语言子集，包括：

- 函数定义、参数、返回类型
- 变量声明、赋值、表达式
- `if / while / for / loop`
- 数组、元组、引用与解引用

输出

- Token 序列：带类别、文本、行列号
- AST：把程序组织成树形结构
- 词法/语法错误定位

对应文件：`token.rs`, `lexer.rs`, `parser.rs`, `ast.rs`

词法分析：从字符流到 Token

- 手写扫描器，逐字符推进，并维护当前位置。
- 先按首字符分类，再对二字符符号做前瞻。
- 标识符先整体读入，再查关键字表。
- 注释、空白符不进入最终 Token 序列。

典型区分

=	赋值
==	相等比较
.	元组字段访问
..	区间表达式
-	减号
->	函数返回类型

词法扫描决策表

首字符/模式	扫描动作	Token 结果
字母或 _	连续读入字母、数字、下划线，再查关键字表	关键字或 Ident(String)
数字	连续读入数字并转为整数	IntLiteral(i64)
= ! < > - .	前瞻一个字符，区分单字符和双字符符号	= == != <= >= -> . ..
/	前瞻判断注释或除号	跳过注释或生成 Slash
空白符	更新行列号，不生成 Token	无
其他字符	生成词法错误，带行列号	错误信息

Token 设计：枚举直接表达语法类别

```
pub enum TokenKind {  
    Fn, Let, Mut, If, Else, While, For, In, Loop,  
    Break, Continue, Return, I32,  
    IntLiteral(i64),  
    Ident(String),  
    Assign, Plus, Minus, Star, Slash,  
    Eq, Ne, Lt, Le, Gt, Ge,  
    Ampersand,  
    LParen, RParen, LBrace, RBrace, LBracket, RBracket,  
    Semicolon, Colon, Comma,  
    Arrow, Dot, DotDot,  
    Eof,  
}
```

设计点：整数和标识符直接携带值，后续 Parser 不需要再查原始字符串。

语法分析：递归下降 + 优先级分层

递归下降

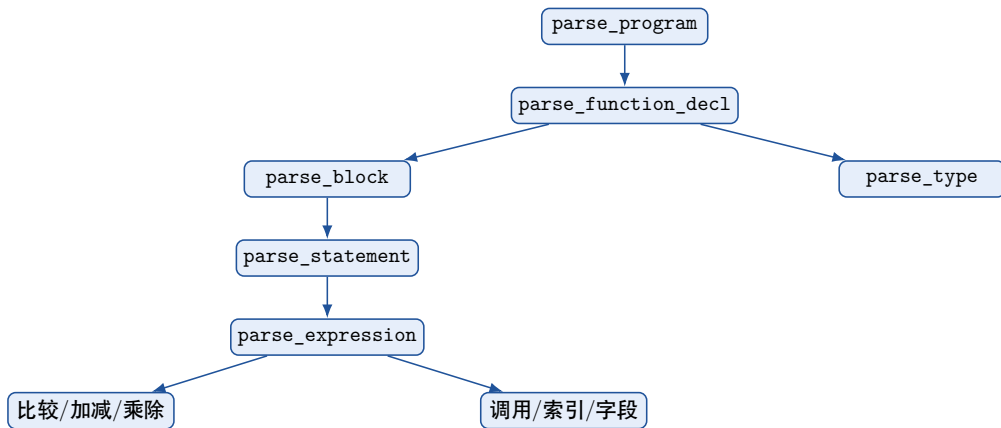
每个非终结符对应一个解析函数，按当前 Token 选择产生式，成功后推进 Token 指针。

- `parse_program`
- `parse_function_decl`
- `parse_statement`
- `parse_expression`

表达式优先级

低	区间 ..
	比较 < <= == !=
	加减 + -
	乘除 * /
	一元 *x, &x
高	调用 / 索引 / 元组访问

Parser 调用层次



结构解析和表达式解析分层：语句层负责控制流和声明，表达式层负责优先级和操作数结合。

AST: 把语法结构保留下来

```
fn add(mut a:i32, b:i32) -> i32 {  
    let mut c = a + b;  
    return c;  
}
```

- 顶层是 Program
- 函数是 FunctionDecl
- 语句是 Statement
- 表达式是 Expr
- 二元运算保留左右子树和操作符

为什么需要 AST

后续语义分析和 IR 生成不再处理字符或 Token，而是遍历结构化的 AST。

AST 节点分类

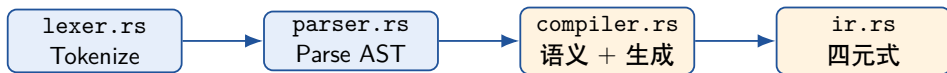
节点族	典型节点	保留的信息
顶层结构	Program, Declaration	函数声明列表
函数结构	FunctionDecl, Param, Block	函数名、参数、返回类型、语句块
语句	VarDecl, Assign, If, While, Return	控制流、声明、赋值和返回
表达式	BinaryOp, Call, Index, Ref, Deref	操作符、操作数、调用参数、访问路径
类型	I32, Array, Tuple, Ref, MutRef	语义分析需要的静态类型

大作业 1 数据流



错误类型	发现位置	例子
非法字符	Lexer	\$
Token 不匹配	Parser	缺少 ; 或 }
表达式结构错误	Parser	括号不匹配、操作数缺失

大作业 2 在大作业 1 基础上的扩展



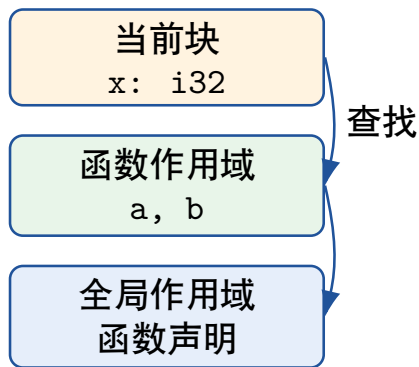
输入	新增状态	输出
AST	符号表栈、循环栈、返回类型	语义错误列表或 IR

符号表：用作用域栈模拟变量生命周期

```
struct SymbolInfo {  
    mutable: bool,  
    ty: Type,  
    arg: Arg,  
}  
  
struct Compiler {  
    scopes: Vec<HashMap<String, SymbolInfo>>,  
    errors: Vec<String>,  
}
```

- 进入函数、代码块、表达式块时
push_scope
- 退出时 pop_scope
- 查询变量时从内层向外层倒序查找
- 支持变量重影，符合类 Rust 的直觉

作用域查询：从内到外



操作	动作	目的
进入块 声明变量 使用变量	push_scope() 插入当前表 反向查找	隔离局部变量 记录类型/可变性 支持遮蔽和外层引用
退出块	pop_scope()	结束生命周期

语义检查覆盖点

错误类型	触发场景	检查位置
未声明变量	<code>a = 1;</code> 但没有 <code>let a</code>	<code>lookup_variable</code>
不可变变量赋值	<code>let a = 1; a = 2;</code>	<code>compile_lvalue</code>
赋值类型不匹配	左值类型与右值类型不同	<code>compile_statement</code>
运算类型不匹配	<code>i32 + tuple</code>	<code>compile_expression</code>
返回类型不一致	<code>-> i32</code> 但 <code>return;</code>	<code>Statement::Return</code>
非法 <code>break/continue</code>	不在循环体内使用	<code>loop_stack</code>

语义错误示例

```
fn test_immutable_assign() {  
    let a: i32 = 1;  
    a = 2;  
}  
  
fn test_break_outside_loop() {  
    break;  
    continue;  
}
```

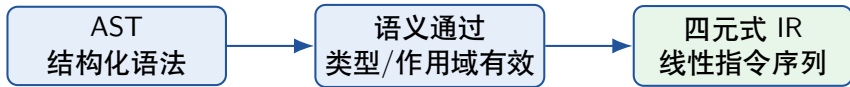
检查逻辑

- 赋值左侧必须是合法左值。
- 左值如果来自不可变变量，则报错。
- break/continue 必须能在 loop_stack 中找到循环上下文。

语义分析状态表

状态	类型	用途
scopes current_ return_type loop_stack	Vec<HashMap<...>> Option<Type> 标签栈	变量声明、类型、可变性和作用域 检查 return 是否匹配函数签名 判断 break/continue 合法性并生成跳转目标
errors	Vec<String>	汇总语义错误，避免只报第一个错误
temp_count label_count	计数器	分配临时变量和控制流标签

第三部分：四元式 IR



(op, arg1, arg2, result)

语法结构	IR 处理	关键状态
表达式	临时变量承接中间结果	temp_count
分支/循环	标签 + 条件/无条件跳转	label_count
break/continue	跳转到循环边界标签	loop_stack

四元式数据结构

```
pub enum Op {  
    Add, Sub, Mul, Div,  
    Eq, Ne, Lt, Le, Gt, Ge,  
    Assign,  
    Jump, JumpIfFalse, Label,  
    Param, Arg, Call, Return,  
    Ref, Deref, DerefAssign,  
    LoadArray, StoreArray,  
}
```

```
pub enum Arg {  
    Empty,  
    Int(i64),  
    Var(String),  
    Temp(usize),  
    Label(usize),  
}  
  
pub struct Quadruple {  
    op: Op,  
    arg1: Arg,  
    arg2: Arg,  
    result: Arg,  
}
```

设计点：表达式结果统一放进临时变量 $t_0, t_1, \dots$ ，控制流目标统一用标签 $L_0, L_1, \dots$ 。

IR 指令族

类别	指令	用途
算术/比较 EQ NE LT LE GT GE	ADD SUB MUL DIV 表达式计算, 结果写入临时变量	
数据移动 DEREF_ASSIGN LOAD_ARRAY	ASSIGN REF DEREF 变量赋值、引用、数组读取	
控制流 JUMP_IF_FALSE	LABEL JUMP 表示分支、循环、break/continue	
函数	PARAM ARG CALL RETURN	参数、调用和返回

Statement 到 IR 的映射

AST 语句	生成逻辑	主要 IR
变量声明赋值	先翻译右值, 再声明变量并赋值	ASSIGN
普通赋值	检查左值可写, 再翻译右值	ASSIGN / Deref_ASSIGN
返回语句	检查返回类型, 发射返回指令	RETURN
if/else	条件失败跳转到 else, then 结束跳到 end	JUMP_IF_FALSE, JUMP, LABEL
while/loop	建立起止标签, 循环体后跳回开始	LABEL, JUMP, JUMP_IF_FALSE

表达式翻译：递归生成临时变量

```
let mut c = a + b * 2;
```

```
(MUL,    b,    2,    t0)
(ADD,    a,    t0,   t1)
(ASSIGN, t1,    -,    c)
```

```
b * 2    ⇒    t0
a + t0    ⇒    t1
t1        ⇒    c
```

if 翻译: 条件跳转 + 两个标签

```
if a > 0 {  
    return 1;  
} else {  
    return 0;  
}
```

```
(GT,          a, 0, t0)  
(JUMP_IF_FALSE, t0, -, L0)  
(RETURN,      1, -, -)  
(JUMP,        -, -, L1)  
L0:  
(RETURN,      0, -, -)  
L1:
```

转换形态: 条件失败跳转 → then → 跳过 else → else 标签。

while/loop 翻译：循环栈管理 break 和 continue

```
let start_label = self.new_label();  
let end_label = self.new_label();  
  
emit(Label,      -,      -, Label(start_label));  
let cond_arg = compile_expression(cond);  
emit(JumpIfFalse, cond_arg, -, Label(end_label));  
  
loop_stack.push((start_label, end_label, None));  
compile_block(body);  
loop_stack.pop();  
  
emit(Jump,      -,      -, Label(start_label));  
emit(Label,     -,      -, Label(end_label));
```

- continue 跳到 start_label, 重新判断循环条件。
- break 跳到 end_label, 离开循环。

循环控制标签

循环入口

start_label

- while: 重新判断条件
- loop: 直接继续下一轮
- continue: 跳到这里

loop_stack.push((start_label, end_label, None))

循环出口

end_label

- 条件失败跳到这里
- break 跳到这里
- 循环结束后的下一条指令

函数调用翻译：参数先入 IR

```
foo(a, 1 + b)
```

```
(ARG,  a,  -, -)
(ADD,  1,  b, t0)
(ARG,  t0,  -, -)
(CALL, foo, 2, t1)
```

字段	含义	示例
arg1	函数名	foo
arg2	参数个数	2
result	返回临时变量	t1

测试与结果

大作业 1

- 内置测试覆盖函数定义、声明、表达式、控制流等规则。
- 输出 Token 序列与 AST。
- 错误测试覆盖非法字符、语法错误等。

大作业 2

- 功能测试生成四元式。
- 语义错误测试覆盖未声明变量、类型不匹配、不可变赋值等。
- 两个 Rust 工程均可通过 cargo check。

测试矩阵

测试文件	覆盖内容	观察输出
test_all.rs.txt	函数、声明、表达式、分支、循环、数组、元组	Token、AST、IR 序列
test_errors.rs.txt	词法/语法错误	带行列号的错误信息
test_semantic_errors.rs.txt	未声明变量、类型不匹配、不可变赋值、非法控制流	[Semantic Error] 列表
cargo check	Rust 工程可编译性	两个工程均无编译错误

总结

最终完成的编译流程

源码 → Token → AST → 语义检查 → 四元式 IR

- 大作业 1 完成前端基础：词法分析、递归下降语法分析和 AST 构造。
- 大作业 2 在 AST 上继续做静态语义检查，并生成四元式中间代码。
- 技术重点在三个衔接：Token 到 AST、AST 到语义信息、结构化控制流到线性 IR。

谢谢老师，欢迎提问

备用：如果问“大作业 1 和 2 有什么区别？”

项目	输入	核心状态	输出
大作业 1	源代码字符串	Token 指针、AST 构造器	Token + AST
大作业 2	AST	符号表栈、循环栈、临时变量计数	语义错误或四元式 IR

备用：为什么选择递归下降？

- 课程给定的类 Rust 子集规模适中，手写递归下降足够清晰。
- 每类语法结构对应一个函数，便于定位错误和展示实现。
- 表达式优先级可以通过多层函数自然表达。
- 不依赖 parser generator，项目文件更少，汇报时更容易解释。

备用：为什么需要语义分析？

- 语法只能判断结构是否匹配，不能判断名字是否声明、类型是否一致。
- 例如 `a = 1`；在语法上像赋值语句，但如果 `a` 没有声明，语义上是错的。
- 例如 `let a = 1; a = 2`；语法没问题，但违反不可变变量不能二次赋值。
- 所以语义分析要维护符号表、类型信息和控制流上下文。

备用：四元式有什么好处？

- 统一表达赋值、算术、比较、跳转、函数调用等操作。
- 复杂表达式会被拆成若干条三地址形式，便于后续优化。
- 控制流被压平成标签和跳转，后续可以进一步映射到汇编或虚拟机指令。
- 对课程项目来说，四元式比直接生成机器码更适合展示编译中间阶段。

备用：现有限制

- 项目重点是课程要求的前端、语义检查和 IR 展示，没有继续生成真实机器码。
- 部分复杂类型和函数签名检查采用简化实现，例如函数调用默认返回 `i32`。
- 数组写入、元组访问等 IR 仍可进一步扩展为更完整的地址计算。
- 这些限制不影响主线：词法/语法分析、语义检查、基础四元式生成已经贯通。